



UNIVERSIDAD DEL VALLE



LG14 - INVESTIGACIÓN Y DESPLIEGUE DE UN REPOSITORIO BIG DATA CON DOCKER

Grupo 5

Integrantes: Liz Gabriela Candia Escobar
Monserrat Salazar Moring
Nayara Kate Hurtado Barja



1. Objetivo

Identificar, analizar y desplegar un repositorio público de GitHub relacionado con **Big Data y tecnologías de procesamiento distribuido**, tomando como referencia el proyecto:

<https://github.com/big-data-europe/docker-hadoop>

El grupo deberá demostrar que comprende la arquitectura del proyecto seleccionado y que es capaz de ejecutarlo y realizar una prueba funcional.

2. Consigna principal

Cada grupo deberá **buscar y seleccionar UN repositorio público de GitHub** que cumpla las siguientes condiciones:

- Estar relacionado con tecnologías Big Data.
- Utilizar Docker, Docker Compose o contenedores.
- Contener archivos de configuración que permitan desplegar el entorno.
- Tener documentación suficiente para comprender su instalación.
- Permitir realizar al menos una prueba funcional.
- **Ser diferente al repositorio seleccionado por los demás grupos.**

No se permite

- Utilizar exactamente el mismo repositorio que otro grupo ya eligió (**RESERVADO O INDICADO EN TEAMS O WHATSAPP**).
- Presentar únicamente una investigación teórica.
- Copiar el README del repositorio como informe.
- Presentar un repositorio sin haberlo ejecutado.
- Cambiar solamente el nombre de una carpeta o hacer una copia/fork y presentarlo como proyecto diferente.

3. Registro obligatorio del repositorio

Antes de comenzar la implementación, cada grupo deberá registrar su repositorio mediante la siguiente información:

Nombre	de	los	estudiantes:
Nombre		del	repositorio:
URL:			
Autor/organización:			
Descripción en una línea:			

Ejemplo:

Nombre de los estudiantes:	Juan Pérez, Edson Coca, Efrain Luna
Repositorio:	XYZ



URL:

<https://github.com/usuario/proyecto>

Descripción: Entorno Docker Compose para ejecutar Apache Spark con almacenamiento distribuido.

Importante

Si dos estudiantes proponen el mismo repositorio, tendrá prioridad el primero que lo haya registrado correctamente.

4. Información que debe analizarse

4.1 Información general

- Autor/Organización: crs4
- Fecha de creación: primer commit: 15 feb 2019 (confírmalo en GitHub)
- Última actualización: último commit: 17 dic 2020
- Estrellas / forks: 5 / 4
- Licencia: Apache-2.0
- Objetivo: imágenes Docker de Apache Hadoop, con un contenedor por servicio.

4.2 Tecnologías

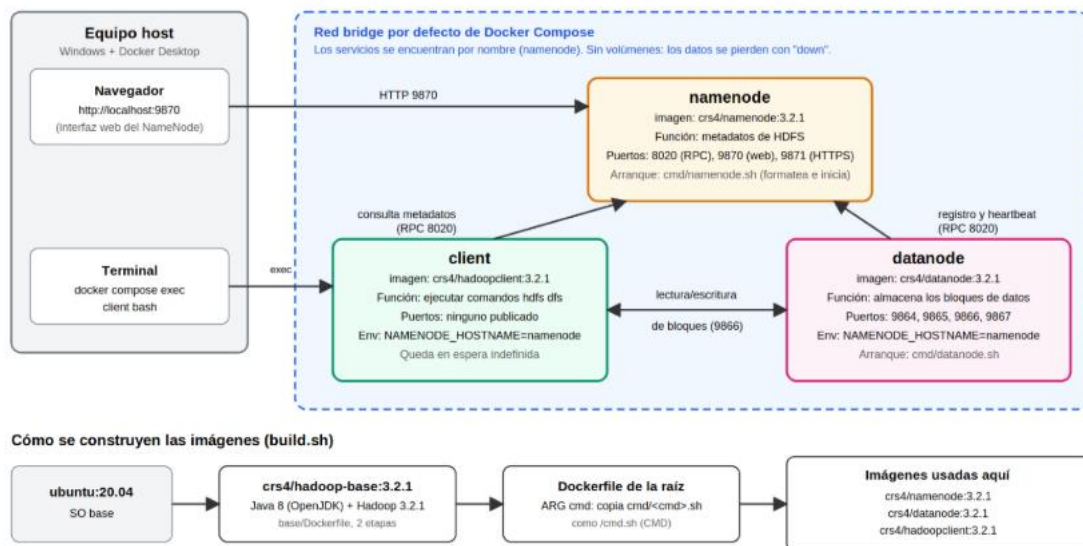
- Tecnología principal: Apache Hadoop 3.2.1 (HDFS y YARN).
- Sistema operativo base: Ubuntu 20.04, con OpenJDK 8 JRE headless.
- Bases de datos: ninguna.
- Lenguajes: Shell 75.6 %, Dockerfile 13.7 %, Python 10.7 %.
- Docker y Docker Compose: anota las versiones con `docker --version` y `docker compose version`.

5. Análisis de la arquitectura

- **Número de contenedores:** 3 (namenode, datanode y client).
- **Nombre de cada contenedor:** namenode, datanode y client. Docker les agrega un prefijo con el nombre de la carpeta; confirma los nombres exactos con `docker ps`.
- **Función de cada contenedor:**
 - namenode: nodo maestro de HDFS, guarda los metadatos del sistema de archivos.
 - datanode: almacena los bloques de datos.
 - client: contenedor de apoyo para ejecutar comandos hdfs; no ejecuta ningún servicio.
- **Imagen Docker utilizada:** crs4/namenode:3.2.1, crs4/datanode:3.2.1 y crs4/hadoopclient:3.2.1, todas basadas en crs4/hadoop-base:3.2.1 (Ubuntu 20.04, OpenJDK 8 y Hadoop 3.2.1).
- **Puertos:**
 - namenode: 8020 (RPC), 9870 (web) y 9871 (web HTTPS).
 - datanode: 9864, 9865, 9866 y 9867.



- client: ninguno.
- **Redes:** no define ninguna, Docker Compose crea una red bridge por defecto y los contenedores se comunican por el nombre del servicio.
- **Volúmenes:** ninguno, los datos se guardan dentro de los contenedores y se pierden con docker compose down.
- **Dependencias entre servicios:** no usa depends_on; el datanode y el client dependen del namenode, al que localizan con namenode_hostname=namenode.
- **Variables de entorno:** namenode_hostname=namenode (en datanode y client), que entrypoint.sh usa para reemplazar localhost en core-site.xml.
- **Archivos de configuración:** docker-compose-hdfs.yml, Dockerfile, base/entrypoint.sh, cmd/*.sh, y en base/resources/conf/hadoop3/ los archivos core-site.xml y hdfs-site.xml (replicación 1).



7. Implementación práctica

El estudiante deberá ejecutar el proyecto.

Como mínimo deberá demostrar:

Paso 1

Clonar el repositorio:

```
git clone URL_DEL_REPOSITORIO
```

Paso 2

Ingresa al directorio:

```
cd NOMBRE_DEL_REPOSITORIO
```

Paso 3



Identificar los archivos principales:

ls -la

```
Salazar@LAPTOP-OEQK70BE MINGW64 /d/Universidad/6. Sexto semestre/TecnologiasEmergentes/HadoopRepo/hadoop-docker (master)
$ ls -la
total 54
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 ./
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 ../
-rw-r--r-- 1 Salazar 197121  77 Oct  4 16:38 .dockerignore
-rw-r--r-- 1 Salazar 197121 149 Oct  4 16:38 .env
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 .git/
-rw-r--r-- 1 Salazar 197121 979 Oct  4 16:38 .travis.yml
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 base/
-rwxr-xr-x 1 Salazar 197121 650 Oct  4 16:38 build.sh*
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 cmd/
-rw-r--r-- 1 Salazar 197121 1214 Oct  4 16:38 docker-compose.yml
-rw-r--r-- 1 Salazar 197121 582 Oct  4 16:38 docker-compose-hdfs.yml
-rw-r--r-- 1 Salazar 197121 141 Oct  4 16:38 Dockerfile
-rw-r--r-- 1 Salazar 197121 428 Oct  4 16:38 Dockerfile.secdn
-rw-r--r-- 1 Salazar 197121 11558 Oct  4 16:38 LICENSE
-rwxr-xr-x 1 Salazar 197121 1202 Oct  4 16:38 manual_build.py*
-rw-r--r-- 1 Salazar 197121 789 Oct  4 16:38 push.sh*
-rw-r--r-- 1 Salazar 197121 3646 Oct  4 16:38 README.md
drwxr-xr-x 1 Salazar 197121  0 Oct  4 16:38 test/

Salazar@LAPTOP-OEQK70BE MINGW64 /d/Universidad/6. Sexto semestre/TecnologiasEmergentes/HadoopRepo/hadoop-docker (master)
$
```

Paso 4

Analizar el archivo de Docker Compose:

cat docker-compose.yml

o el archivo equivalente utilizado por el proyecto.



```
Salazar@LAPTOP-OEQK70BE MINGW64 /d/Universidad/6. Sexto semestre/TecnologiasEmergentes/HadoopRepo/hadoop-doc
ker (master)
$ ls -la
total 54
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 ./
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 ../
-rw-r--r-- 1 Salazar 197121  77 Oct 4 16:38 .dockerignore
-rw-r--r-- 1 Salazar 197121 149 Oct 4 16:38 .env
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 .git/
-rw-r--r-- 1 Salazar 197121 979 Oct 4 16:38 .travis.yml
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 base/
-rwxr-xr-x 1 Salazar 197121 650 Oct 4 16:38 build.sh*
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 cmd/
-rw-r--r-- 1 Salazar 197121 1214 Oct 4 16:38 docker-compose.yml
-rw-r--r-- 1 Salazar 197121 502 Oct 4 16:38 docker-compose-hdfs.yml
-rw-r--r-- 1 Salazar 197121 141 Oct 4 16:38 Dockerfile
-rw-r--r-- 1 Salazar 197121 428 Oct 4 16:38 Dockerfile.secdn
-rw-r--r-- 1 Salazar 197121 11558 Oct 4 16:38 LICENSE
-rwxr-xr-x 1 Salazar 197121 1202 Oct 4 16:38 manual_build.py*
-rwxr-xr-x 1 Salazar 197121 789 Oct 4 16:38 push.sh*
-rw-r--r-- 1 Salazar 197121 3646 Oct 4 16:38 README.md
drwxr-xr-x 1 Salazar 197121  0 Oct 4 16:38 test/

Salazar@LAPTOP-OEQK70BE MINGW64 /d/Universidad/6. Sexto semestre/TecnologiasEmergentes/HadoopRepo/hadoop-doc
ker (master)
$
```

Paso 5

Construir y ejecutar:

`docker compose up -d`

```
$ cat docker-compose-hdfs.yml
# HDFS cluster. Port values are Hadoop3-specific.

version: "3"

services:
  namenode:
    image: crs4/namenode:3.2.1
    ports:
      - "8020:8020"
      - "9870:9870"
      - "9871:9871"

  datanode:
    image: crs4/datanode:3.2.1
    ports:
      - "8054:8054"
      - "8060:8060"
      - "8065:8065"
      - "8067:8067"
    environment:
      - NAMESPACE_HOSTNAME=namenode

  client:
    image: crs4/hadoopclient:3.2.1
    environment:
      - NAMESPACE_HOSTNAME=namenode
```



Paso 6

Verificar los contenedores:

docker ps

```
PS D:\Universidad\6. Sexto semestre\TecnologíasEmergentes\Vscode\bigdata-grupo-5> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
8cf135b66ee   crs4/datanode:3.2.1                "/entrypoint.sh /cmd..." 2 minutes ago  Up 2 minutes  0.0.0.0:9864-9867->9864-9867/tcp, [...] 9864-9867->9864-9867/tcp  bigdata-grupo-5-datanode-1
1b3698c8a65a   crs4/namenode:3.2.1                "/entrypoint.sh /cmd..." 2 minutes ago  Up 2 minutes  0.0.0.0:8020->8020/tcp, [...] 8020->8020/tcp, 0.0.0.0:9870-9871->9870-9871/tcp, [...] 9870-9871->9870-9871/tcp  bigdata-grupo-5-namenode-1
ff8bed02f2fd   crs4/hadoopclient:3.2.1            "/entrypoint.sh /cmd..." 2 minutes ago  Up 2 minutes  22192433c074   alpine                  "sh -c 'while true; ..." 11 days ago           Up 10 minutes
PS D:\Universidad\6. Sexto semestre\TecnologíasEmergentes\Vscode\bigdata-grupo-5>
```

Paso 7

Realizar una prueba funcional relacionada con la tecnología seleccionada.

```
PS D:\Universidad\6. Sexto semestre\TecnologíasEmergentes\Vscode\bigdata-grupo-5> docker compose -f docker-compose-hdfs.yml exec client bash
root@ff8bed02f2fd:/# echo "hola bigdata" > prueba.txt
root@ff8bed02f2fd:/# hdfs dfs -put -f prueba.txt /user/prueba/
2026-10-04 21:12:40,807 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
root@ff8bed02f2fd:/# hdfs dfs -ls /user/prueba
Found 1 items
-rw-r--r-- 1 root supergroup 13 2026-10-04 21:12 /user/prueba/prueba.txt
root@ff8bed02f2fd:/# hdfs dfs -cat /user/prueba/prueba.txt
2026-10-04 21:12:48,354 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
hola bigdata
root@ff8bed02f2fd:/#
```



7. Prueba funcional obligatoria

La prueba deberá estar relacionada directamente con la tecnología principal.

Hadoop / HDFS

Demostrar:

- Creación de un directorio.
- Carga de un archivo.
- Consulta del archivo almacenado.

8. Comparación con docker-hadoop

El estudiante deberá comparar su proyecto con:

Big Data Europe – Docker Hadoop

<https://github.com/big-data-europe/docker-hadoop>

La comparación deberá incluir:

Característica	docker-hadoop	Repositorio seleccionado
Tecnología principal	Hadoop	Hadoop
Docker	Sí	Si
Docker Compose	Sí	Si
Número de contenedores	5: namenode, datanode, resourcemanager, nodemanager, historyserver	3: namenode, datanode, client (compose HDFS).
Almacenamiento distribuido	HDFS (NameNode + DataNode)	HDFS con 1 DataNode
Procesamiento distribuido	YARN y MapReduce	No en nuestro despliegue (solo HDFS). El compose completo incluye YARN y MapReduce, pero no las levantamos



Interfaces web	5: NameNode 9870, DataNode 9864, ResourceManager 8088, NodeManager 8042, HistoryServer 8188	2: NameNode (9870, HTTPS 9871) y DataNode (9864, HTTPS 9865)
Persistencia	Volúmenes de Docker para datos del NameNode, DataNode e HistoryServer	Ninguna: no define volúmenes, los datos se pierden con docker compose down
Complejidad de instalación	Baja: docker-compose up con imágenes ya publicadas	Baja-media: pull y up; si las imágenes no están, hay que construirlas con build.sh
Documentación	README breve: arranque rápido, URLs y configuración por variables en hadoop.env	README breve: construcción, uso, cambio de versión y configuración propia
Caso de uso	Clúster HDFS/YARN de ejemplo y base para otros stacks	Imágenes Hadoop personalizables, una por servicio, para pruebas y desarrollo

El objetivo **no es determinar cuál es mejor**, sino identificar las diferencias de arquitectura, tecnologías y propósito.

9. Criterio fundamental de la actividad

La actividad será evaluada principalmente sobre la capacidad del grupo para:

"Encontrar → comprender → desplegar → probar → analizar"

No se evaluará únicamente la capacidad de encontrar un repositorio interesante.

El grupo deberá demostrar que **comprende qué implementó y cómo funciona**.

10. Entrega de trabajo en github

Todo el trabajo deberá ser entregado mediante un **repositorio propio de GitHub**.

El repositorio deberá contener como mínimo:

- **README.md** con la descripción del proyecto, instalación, ejecución y resultados.



- Archivos de configuración utilizados (`docker-compose.yml`, Dockerfile, etc.).
- Scripts utilizados.
- Diagrama de arquitectura.
- Evidencias de la implementación y pruebas.

Requisitos

- El repositorio deberá ser **público**.
- El nombre deberá identificar al grupo, por ejemplo:
`bigdata-numero_de_grupo`
- Deberá incluir al menos **3 commits** que evidencien el avance del trabajo.

Entrega

El estudiante deberá entregar únicamente la **URL de su repositorio GitHub**.